Schema Convergence Through Views and Transforms

Tom Tang
Edge Vector Foundation

2026

Abstract

Users store data in personal schemas that reflect the idiosyncratic structure of their sources—contact lists, health records, financial exports, social media archives. Despite surface-level differences, these schemas encode the same underlying concepts. We present a system in which *views*—content-addressed WebAssembly transforms over queries—unify heterogeneous user schemas into canonical output schemas without requiring users to restructure their data. Views are registered alongside schemas in a shared registry, deterministically producing output whose schema is known at registration time. When a view's transform is reversible, writes to the output propagate back to the source; when irreversible, source fields are marked stale. A micropayment layer incentivizes schema creation and reuse: every query against another user's schema triggers a payment to the data owner. This creates a market dynamic in which application developers build views that compose many source schemas into unified APIs, and data owners are compensated each time their data is accessed. Over time, the combination of economic incentives and composable views drives schema convergence across the network without centralized coordination.

1 Introduction

The promise of user-owned data founders on a practical problem: heterogeneity. When each user controls their own database, there is no DBA enforcing a canonical schema. One user's contact list has `full_name`; another has `first_name` and `last_name`. A health app exports `weight_kg`; a fitness tracker stores `weight_lbs`. A social media archive uses `created_at` as a Unix timestamp; another uses ISO 8601.

Conventional approaches to this problem—ETL pipelines, schema migration scripts, or centralized canonical models—all require a coordinating authority. They assume someone can define the "right" schema and force adoption. In a decentralized, user-owned data system, no such authority exists.

We observe that the problem is not schema *disagreement* but schema *translation*. The underlying data is semantically equivalent; only the surface representation differs. If the system provides a lightweight, composable mechanism for expressing translations, users and application developers will create them organically—especially if there is an economic incentive to do so.

This paper presents that mechanism: *views* backed by WebAssembly transforms, registered in a content-addressed view registry alongside the existing schema registry. We show how views enable schema convergence without coordination, how bidirectional transforms maintain consistency, and how micropayments create a self-reinforcing cycle of schema creation and reuse.

2 System Architecture

2.1 Schema Registry (Review)

In FoldDB, every schema is content-addressed: its identity is the hash of its structural definition (field names, types, and classifications). Two users who independently create identical schemas produce the same hash and are automatically recognized as compatible. Schemas are the mechanism for *direct writes*—data enters the system through a schema.

Definition 1 (Schema)**.** *A schema S is a tuple (F, T, C, h) where F is an ordered set of field names, T maps each field to a type, C maps each field to a set of classification labels, and $h = \mathcal{H}(F, T, C)$ is the content hash serving as the schema's identity.*

2.2 View Registry

We introduce a parallel *view registry*. A view is a named, content-addressed transform that takes one or more queries as input and produces JSON output conforming to a declared output schema.

Definition 2 (View)**.** *A view V is a tuple (Q, W, S_{out}, h_V) where:*

- *$Q = \{q_1, \ldots, q_n\}$ is a set of input queries, each specifying a source schema and dataset,*
- *W is a compiled WebAssembly module implementing the transform,*
- *S_{out} is the output schema (a standard schema from the schema registry),*
- *$h_V = \mathcal{H}(Q, W, S_{out})$ is the view's content hash.*

The view's content hash ensures that two developers who independently write the same transform over the same inputs, producing the same output schema, are recognized as equivalent. This is identical to how schema identity works—the registry deduplicates automatically.

2.3 WebAssembly Transform Layer

The transform W is a WebAssembly module with a minimal interface:

```
// WASM module interface
fn transform(inputs: &[JSON]) -> JSON
```

JSON in, JSON out. The WASM module receives the results of the input queries as a JSON array and returns a single JSON object conforming to S_{out}. The runtime enforces that:

1. The output is valid JSON.
2. The output conforms to S_{out} (all required fields present, types correct).
3. The module executes within bounded time and memory (WASM sandboxing).
4. The module has no side effects (pure function).

Using WebAssembly provides language-agnostic authoring (developers can write transforms in Rust, Go, AssemblyScript, or any language targeting WASM), deterministic execution across all nodes, sandboxed isolation with no access to the host system, and near-native performance.

2.4 Execution Model

When a user or application queries a view:

1. The runtime resolves the view's input queries Q against the schema registry.
2. Each input query executes against the user's data, producing JSON results. Access policies (trust distance, capabilities, payment gates) are enforced at this stage.
3. The input results are passed to the WASM transform W.
4. The transform produces output JSON, which is validated against S_{out}.
5. The validated output is returned to the caller.

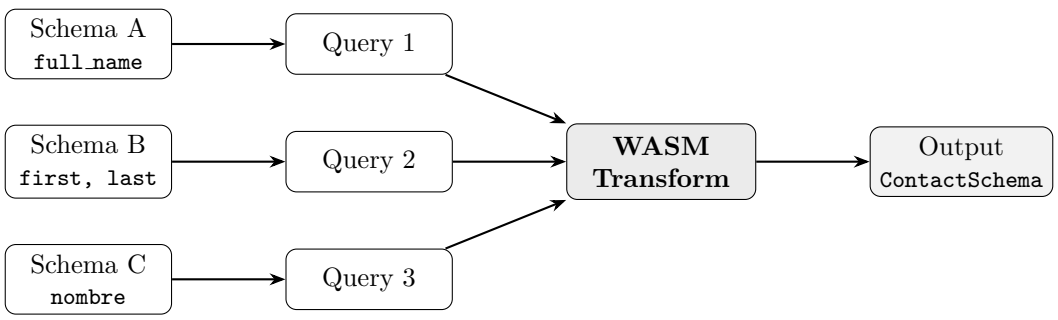

Figure 1: A view unifies three heterogeneous contact schemas into a single canonical output schema via a WASM transform.

3 Schema Convergence

3.1 The Convergence Dynamic

Schema convergence does not require that users change their data. It requires only that views exist to translate between representations. Convergence emerges from three forces:

1. Content addressing creates natural attractors. When two users independently create schemas with identical structure, the schema registry recognizes them as the same schema (identical content hash). This is "convergence at zero cost"—it happens automatically for any users whose data sources produce the same shape.

2. Views bridge the remaining gaps. For schemas that are semantically equivalent but structurally different, views provide the translation layer. A single view can map N source schemas to one output schema. Once a view exists, any user whose data matches one of the input schemas gains access to the output schema's ecosystem.

3. Economic incentives drive view creation. Application developers need unified data across users to build useful products. The view mechanism lets them define exactly the unified schema they need and write transforms to reach it. The micropayment system (Section 5) ensures that data owners are compensated, creating a positive-sum dynamic.

3.2 Convergence Over Time

We model convergence as a process on a bipartite graph $G = (U, A, E)$ where U is the set of user schemas, A is the set of application-defined output schemas, and E is the set of views connecting them. At time t_0, the graph is sparse—few views exist. As developers create views:

1. **Hub formation.** Popular output schemas ("ContactInfo", "HealthRecord", "FinancialSummary") become hubs with many incoming edges. These are the schemas that applications actually query.

2. **Source consolidation.** Users whose idiosyncratic schemas are connected to hubs via views gain the same access as users with the hub schema directly. Over time, new users may adopt hub schemas directly (since they work with the most applications), reducing heterogeneity at the source.

3. **View composition.** Views can query other views' output schemas as inputs. This creates transitive reachability: if view V_1 maps schema A to schema B, and view V_2 maps schema B to schema C, then data in schema A is reachable from schema C through the composition $V_2 \circ V_1$.

Property 1 (Monotonic Reachability)**.** *The set of schemas reachable from any given output schema is monotonically non-decreasing over time. Adding a new view never reduces reachability.*

4 Bidirectional Consistency

A critical question arises: can views be written to? The answer is always yes. What varies is the consistency guarantee.

4.1 Reversible Transforms

Definition 3 (Reversible View)**.** *A view $V = (Q, W, S_{out}, h_V)$ is* reversible *if the WASM module W also exports a function:*

```
fn reverse(output: JSON, original_inputs: &[JSON]) -> Vec<JSON>
```

that, given the output and the original input values, produces updated input values such that re-running W on the updated inputs yields the new output.

When a user writes to a reversible view:

1. The runtime calls `reverse` with the new output value and the current input values.

2. The reverse function produces updated input JSON for each source query.

3. The updates are written back to the source schemas, subject to the source schemas' access policies.

4. The view is re-evaluated to confirm consistency.

Example. A view concatenates `first_name` and `last_name` into `full_name`. The reverse splits on the first space. Writing "Alice Smith" to `full_name` propagates "Alice" to `first_name` and "Smith" to `last_name`.

Example. A unit conversion view transforms `weight_kg` to `weight_lbs` via multiplication by 2.20462. The reverse divides by 2.20462. Writing 150 lbs propagates 68.04 kg back to the source.

4.2 Irreversible Transforms

Many useful transforms are not reversible: aggregations (sum, average, count), hash functions, lossy projections (dropping fields), or many-to-one mappings. For these, the system uses a *staleness protocol*.

When a user writes to a field in an irreversible view's output:

1. The new value is stored as an *override* on the view's output.
2. The source fields that contributed to the overridden output field are marked **stale** in the source schema.
3. A staleness annotation records which view, which output field, and when the override occurred.
4. Subsequent reads of the stale source fields return the original value plus the staleness annotation.

The staleness marker signals to the data owner (and any other views depending on those source fields) that the source and the derived view have diverged. The owner can then choose to:

- Update the source fields manually to be consistent with the view override.
- Discard the view override, restoring the derived value.
- Leave both values as-is, accepting the divergence.

Property 2 (Write Availability)**.** *Every view accepts writes. No view is read-only. The system always accepts new data; what varies is the propagation guarantee (full propagation for reversible, staleness marking for irreversible).*

4.3 Formal Consistency

Let V be a view with transform W, input queries producing values $\vec{x} = (x_1, \ldots, x_n)$, and output $y = W(\vec{x})$.

Reversible case. After a write of y' to the output, the system computes $\vec{x}' = W^{-1}(y', \vec{x})$ and writes $\vec{x}'$ to the sources. The invariant $W(\vec{x}') = y'$ holds by construction (enforced by the re-evaluation step).

Irreversible case. After a write of y' to the output, the system stores (y', t) as an override. For each source field x_i that contributed to the overridden output field, the system records $\text{stale}(x_i, V, t)$. The invariant is: $W(\vec{x}) \neq y'$ and the staleness annotation makes this divergence visible.

5 Micropayments and Economic Incentives

5.1 The Payment Model

FoldDB enforces a micropayment on every data access by a non-owner. When an application queries a user's schema (directly or through a view), the following payment flow executes:

1. The query identifies which schemas and fields are accessed.
2. For each accessed field, the system checks whether the querier is the data owner.
3. If not, a micropayment is debited from the querier's account and credited to the data owner's account.
4. Payment amounts are set per-field by the schema's payment policy (part of the fold access control).

This applies transitively through views: if a view queries three source schemas owned by three different users, a single query against the view triggers three micropayments.

5.2 Incentive Alignment

The micropayment model creates aligned incentives for all participants:

Data owners are incentivized to:

- Store more data (more fields = more potential revenue).
- Use well-known schemas (applications can query them without custom views).
- Approve views that reference their data (views increase the number of applications that can access their data).

Application developers are incentivized to:

- Create views that unify many source schemas into one output schema, because this maximizes the number of users whose data they can access through a single API.
- Publish their views publicly, because a widely-used view becomes a de facto standard and reduces per-query development cost.
- Adopt existing popular output schemas rather than inventing new ones, because existing schemas already have views connecting them to user data.

The network converges because:

- Each new view increases the connectivity of the schema graph.
- Popular output schemas attract more views (more revenue for data owners whose schemas are connected).
- Data owners gravitate toward schemas that are connected to many views (more applications = more micropayment revenue).
- The cost of heterogeneity is borne by the developer who writes the view, not by the user who must restructure their data.

5.3 View Economics

View creators may optionally attach a micropayment to the view itself: a small fee charged to any application that *uses* the view (separate from the data access fees paid to data owners). This creates a market for data integration:

- A developer who writes a high-quality view unifying 50 contact schemas earns a fee each time an application uses that view.
- Competing views over the same source schemas compete on quality, performance, and price.
- The content-addressing of views means that identical transforms are free (same hash = same view, no duplicate fee).

6 Schema Evolution

Traditional schema evolution (adding columns, renaming fields, changing types) is painful because it requires migrating existing data. Views eliminate this problem.

6.1 Non-Breaking Evolution

Suppose an application currently uses output schema S_1 with fields {`name, email`}. The developer wants to add a `phone` field, creating schema S_2 with fields {`name, email, phone`}.

Without views, this is a migration. With views:

1. The developer registers S_2 in the schema registry (new content hash).
2. The developer creates a new view V_2 that queries the same source schemas as V_1 but also queries schemas containing phone data.
3. The WASM transform in V_2 includes the phone field in its output.
4. Applications that upgrade to S_2 use V_2; applications on S_1 continue using V_1.
5. No data is migrated. Both views coexist.

6.2 Deprecation Without Deletion

Old schemas and views are never deleted (they are content-addressed and immutable). They simply accumulate fewer queries over time as applications migrate to newer versions. The micropayment model provides a natural signal: when a schema or view stops generating revenue, it is effectively deprecated.

6.3 Retroactive Unification

A powerful consequence of the view model is *retroactive unification*. Suppose 1,000 users have been storing contact data in 15 different schemas for years. A developer creates a single view that maps all 15 schemas to one output schema. Instantly, all 1,000 users' historical data is accessible through the unified schema—no migration, no data movement, no user action required.

7 Implementation

7.1 View Registration

A view is registered by submitting its WASM module, input query specifications, and output schema to the view registry. The registry:

1. Compiles and validates the WASM module.
2. Verifies that the output schema exists in the schema registry.
3. Computes the content hash $h_V = \mathcal{H}(Q, W, S_{\text{out}})$.
4. If h_V already exists, returns the existing view (deduplication).
5. Otherwise, stores the view and indexes it by input schemas and output schema.

7.2 Reversibility Declaration

When registering a view, the developer optionally declares it as reversible by including a `reverse` export in the WASM module. The registry verifies reversibility by:

1. Generating random input values conforming to the input schemas.
2. Running the forward transform to produce output.
3. Mutating the output.
4. Running the reverse transform on the mutated output.
5. Running the forward transform on the reverse output.
6. Checking that the result matches the mutated output (round-trip property).

This is a probabilistic verification (not a proof), but it catches common errors. Developers can also provide a formal proof of reversibility as metadata.

7.3 Query Planning

When an application queries an output schema, the runtime must determine which view to use. The query planner:

1. Looks up all views whose output schema matches the query target.
2. For each candidate view, checks whether the current user has data matching the input queries.
3. Ranks candidates by: (a) number of input schemas matched, (b) view freshness, (c) view fee.
4. Selects the best candidate and executes.

If no view directly produces the target schema, the planner searches for transitive paths (view composition). This is a graph search over the view registry, bounded by a configurable maximum depth.

7.4 Caching and Materialization

View outputs are ephemeral by default (computed on each query). For frequently-accessed views, the system supports *materialization*: pre-computing and caching the output. Materialized views are invalidated when any source data changes (detected via the source schema's change log). The staleness protocol (Section 4.2) extends to materialized views: a write to a materialized irreversible view marks the source fields as stale, and the materialized cache is updated with the override.

8 Security Considerations

8.1 WASM Sandboxing

All transforms execute in a WebAssembly sandbox with:

- No filesystem, network, or system call access.
- Bounded memory (configurable per-view, default 16MB).
- Bounded execution time (configurable, default 1 second).
- No shared state between invocations (pure function).

A malicious WASM module can waste compute but cannot exfiltrate data, modify state, or escape the sandbox.

8.2 Access Policy Enforcement

Views do not bypass access policies. Each input query in a view's specification is subject to the full access control stack (trust distance, capabilities, payment gates, lattice labels). A view cannot access data that the querier could not access directly. The view is a transform over authorized data, not an escalation mechanism.

8.3 Content-Address Integrity

Because views are content-addressed, a view's behavior is immutable. The WASM module referenced by hash h_V always produces the same output for the same input. There is no mechanism for a view to be silently modified after registration. A new version of a view is a new view with a new hash.

9 Related Work

Database views (SQL `CREATE VIEW`) provide read-only derived relations but lack content addressing, economic incentives, and bidirectional write propagation. Materialized views add caching but not portability across heterogeneous schemas.

GraphQL and similar API layers address schema unification at the application boundary but require a centralized server that understands all source schemas. Our approach pushes the translation into the data layer itself, where it composes with access control and micropayments.

Schema registries (Confluent Schema Registry, AWS Glue Schema Registry) provide versioning and compatibility checking but assume a centralized authority that defines the canonical schema. Our registry is append-only and content-addressed, with no authority—compatibility is emergent.

WASM-based plugin systems (Envoy filters, Spin components, Extism) demonstrate the viability of sandboxed, language-agnostic computation at the data path. We apply the same pattern to data transforms within a database.

Lens-based bidirectional programming [1] formalizes the theory of reversible transforms. Our reversible views are a practical application of this theory, with the irreversible case handled by staleness marking rather than by restricting the transform language.

10 Conclusion

Schema convergence in a decentralized, user-owned data system does not require centralized coordination or forced migration. It requires three things: a content-addressed schema registry that automatically deduplicates identical schemas, a view registry with sandboxed WASM transforms that bridge heterogeneous schemas, and an economic incentive (micropayments) that rewards data owners and view creators for participation.

Views are the key abstraction. They separate the concern of *how data is stored* (the user's choice) from *how data is accessed* (the application's need). The WASM transform layer is deliberately minimal—JSON in, JSON out—making view authoring accessible to any developer. Content addressing ensures that the view ecosystem is self-deduplicating and immutable. Bidirectional writes ensure that views are not second-class: every view accepts writes, with full propagation for reversible transforms and explicit staleness marking for irreversible ones.

The result is a system in which the schema graph grows denser over time, popular output schemas become hubs, and the cost of data heterogeneity is borne by view creators (who are compensated for their work) rather than by data owners (who should not have to restructure their data to participate in the network).

References

[1] A. Bohannon, J. N. Foster, B. C. Pierce, A. Pilkiewicz, and A. Schmitt. Boomerang: Resourceful lenses for string data. In *Proc. POPL*, pages 407–419, 2008.